Design and Analysis of Algorithms

Assignment 1: Problem Statement

Assignment title: Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System
Course outcome: CO1 (algorithm fundamentals, efficiency analysis, asymptotic analysis, mathematical analysis, recurrence analysis, Master Theorem, substitution)
Total marks: 50
Duration: 1 week

1. Background

A university keeps a large academic database in which every student record is identified by a unique Registration Number. Faculty and administrative staff regularly need to do two things: check whether a particular student record exists, and retrieve the information stored against it.

When the database is small, almost any search method works. As it grows, the choice of method decides whether a lookup feels instant or painfully slow. This assignment asks you to study that effect carefully.

2. Problem Description

The database may contain:

1,000 → 10,000 → 100,000 → 1,000,000+ records

You must investigate, design and compare efficient algorithmic approaches for locating a given registration number as the database grows.

Two conditions apply throughout:

The solution must stay practical for both small and very large datasets.

Any preprocessing performed on the records, such as sorting or building a lookup structure, must be included in your efficiency analysis. An algorithm cannot look fast just because its setup cost was left out.

3. Design Challenge (Changed Requirement)

The university now needs to process 50,000 search requests per hour on a database of 1,000,000 student records. New student records are added only once per day.

You must re-evaluate your original design and decide whether to:

RETAIN → MODIFY → or REDESIGN

The reasoning behind your decision matters more than the decision itself. It has to follow from your analysis and your experimental evidence.

4. Objectives

By completing this assignment, you should be able to:

Design more than one algorithm for the same problem and justify why each is worth studying.

Identify the basic operation that drives an algorithm's running time.

Derive time-complexity functions mathematically instead of quoting them.

Explain best-, average- and worst-case behaviour in terms of input conditions.

Use O, Ω and Θ notation and interpret what growth rates mean in a real application.

Formulate a recurrence relation from a recursive algorithm and solve it analytically.

Implement the algorithms and test them on datasets of increasing size.

Compare theoretical predictions with measured behaviour and explain any gap.

Defend a final design decision using evidence, and adapt it when requirements change.

5. Tasks

Task 1: Design the Algorithm (5 marks)

Design and propose at least three meaningfully different algorithmic strategies for locating a given registration number. The strategies must be genuinely distinct. An iterative and a recursive version of the *same* underlying algorithm do not count as two different approaches.

Your proposal must include:

At least one iterative approach

At least one recursive approach

A clear explanation of the basic idea behind each approach

Pseudocode or a flowchart for each approach

A brief justification for choosing each approach for investigation

Any preprocessing the approach needs, such as sorting or restructuring the data

You are expected to choose the approaches yourself. The suitability and diversity of your choices are part of the assessment.

Evidence expected: Three meaningfully different strategies, with pseudocode or flowcharts, the design strategy identified, preprocessing requirements, and a justification for each.

Task 2: Identify the Basic Operation (2 marks)

For each proposed algorithm, identify the basic operation that mainly determines its execution time. Discuss:

What the basic operation is

Where it occurs in the algorithm

Why it is appropriate to use for efficiency analysis

How the number of times it executes changes as the input size n increases

Do not simply name an operation. Relate it to how the algorithm actually behaves.

Evidence expected: Correct identification and justification of the dominant operation for each algorithm.

Task 3: Derive the Time-Complexity Function Mathematically (6 marks)

Derive the time-complexity function of each approach using mathematical analysis. Discuss:

The input size (n)

The number of times the basic operation executes

How that count changes with n

The mathematical expression for the running time

Any preprocessing cost

The overall cost when preprocessing and searching are considered together

Show the reasoning that leads to the result. Merely stating “O(n)” or “O(log n)” will not receive full credit.

Evidence expected: A clear derivation of the time-complexity function for each approach, including operation counts and preprocessing costs.

Task 4: Analyse Best, Average and Worst-Case Behaviour (4 marks)

Analyse how each algorithm performs under different input conditions. Wherever applicable, discuss:

The input condition that produces the best case

The input condition that produces the average case

The input condition that produces the worst case

How the number of operations changes in each case

Why these differences occur

Do not just present a table of complexity values. Explain the conditions that lead to each case.

Evidence expected: A justified best-, average- and worst-case analysis, supported by mathematical reasoning.

Task 5: Perform Asymptotic Analysis (3 marks)

Express and compare the growth behaviour of your algorithms using appropriate asymptotic notation. Discuss:

Appropriate use of O, Ω and/or Θ, wherever relevant

How each algorithm behaves as n becomes very large

How the growth rates of the approaches differ

What this growth behaviour means when the dataset grows from 1,000 to 1,000,000+ records

Interpret the behaviour in the context of the university application rather than only stating the notation.

Evidence expected: Correct asymptotic representation and a meaningful comparison of growth behaviour.

Task 6: Formulate the Recurrence Relation (3 marks)

For the recursive approach, formulate a recurrence relation directly from the behaviour of the algorithm. Clearly identify:

The input size of the original problem

The size of the recursive subproblem

The number of recursive calls

The work performed outside the recursive call

The base case

How these components combine into the recurrence

The recurrence must be derived from your own algorithm, not copied as a known formula.

Evidence expected: A correctly formulated recurrence with its components and base case clearly identified.

Task 7: Solve the Recurrence Relation (4 marks)

Solve the recurrence from Task 6 using an appropriate analytical method. You may use:

The substitution method

The iteration method

The Master Theorem

Another appropriate analytical technique

Discuss:

Why the chosen method is applicable

The steps involved in solving the recurrence

The resulting complexity

What the result says about the recursive algorithm as n increases

Show the analytical process, not only the final answer.

Evidence expected: A complete solution using an appropriate method, with justification of the method and interpretation of the result.

Task 8: Implement and Test the Algorithms (4 marks)

Implement all proposed approaches and test their behaviour experimentally on datasets of approximately:

1,000 → 10,000 → 100,000 → 1,000,000 records

Make sure that:

All algorithms produce the correct result

The same or equivalent datasets are used for comparison

Testing conditions are kept reasonably consistent

Appropriate test cases are included

The implementation corresponds to the algorithms analysed in the earlier tasks

The purpose is not merely to show working code. The implementation exists to generate evidence that validates your theoretical analysis.

Evidence expected: Working implementations and a clearly described, reproducible testing procedure.

Task 9: Record Experimental Performance (3 marks)

Collect and present experimental performance data for each algorithm across the different input sizes. Record suitable measures such as:

Execution time

Number of comparisons

Number of key operations

Input size

Other relevant measures, where justified

Present the results using appropriate tables, graphs and performance plots, so that the approaches can be compared easily.

Evidence expected: Systematically collected experimental data, presented clearly in tables and/or graphs.

Task 10: Compare Theory with Experimental Behaviour (4 marks)

Compare your theoretical analysis with your experimental results. Discuss:

Whether the observed growth agrees with the theoretical complexity

Which algorithm scales better as n increases

Any differences between theoretical prediction and actual execution

Possible reasons for those differences

Interpret the results rather than simply reproducing tables or graphs.

Evidence expected: A reasoned comparison of theoretical and experimentally observed performance.

Task 11: Recommend and Defend the Final Design (4 marks)

Based on your mathematical analysis and experimental findings, recommend the most appropriate solution for the student-record system. Discuss:

Which approach you recommend

Why it is more appropriate than the alternatives

Its theoretical efficiency

Its experimentally observed performance

Any preprocessing cost involved

Relevant trade-offs or limitations

Whether it stays practical as the database grows

Your recommendation must be based on evidence from your own analysis and experiments. Simply naming the algorithm with the lowest Big-O complexity is not sufficient.

Evidence expected: A technically justified recommendation supported by both mathematical analysis and experimental evidence.

6. Day-4 Design Challenge

Task 12: Re-evaluate and Adapt the Design (8 marks)

The operating conditions have changed: 50,000 searches per hour on 1,000,000 records, with new records added only once per day. Re-evaluate your original design and decide whether to retain, modify or redesign it. This task has four parts, worth 2 marks each.

A. Impact Analysis (2 marks)
Discuss what has changed from the original problem, which original assumptions are affected, why the frequency of searches now matters, and why the low frequency of record updates may influence the design.
Evidence expected: A clear analysis of how the changed requirements affect the original design.

B. Evaluation of the Original Design (2 marks)
Evaluate whether your originally recommended solution is still appropriate. Discuss its expected behaviour under 50,000 searches per hour, whether its preprocessing cost is acceptable under the new conditions, and whether another approach could give better overall performance.
Evidence expected: A technically supported evaluation of the original design under the new conditions.

C. Adaptation or Redesign (2 marks)
Decide whether to retain, modify or redesign the original solution. If you modify or redesign it, describe the revised strategy, explain what has changed, and determine the effect on computational complexity. If you retain it, give enough technical justification for why no redesign is needed.
Evidence expected: An appropriate design response to the changed requirement, with clear algorithmic reasoning.

D. Evidence-Based Defence (2 marks)
Defend your final decision using mathematical complexity analysis, experimental results where appropriate, search frequency, preprocessing cost, dataset size and relevant trade-offs. The decision must follow logically from the analysis.

Evidence expected: A well-supported decision that combines theory, experiments and the changed constraints.